TP — Tests unitaires avec JUnit 5

Application Boutique en ligne · Panier & ServiceCommande · JUnit 5

Objectifs du TP

À la fin du TP, vous devez être capables de :

- écrire des tests unitaires avec JUnit 5
- structurer un test avec AAA (Arranger · Agir · Affirmer)
- identifier des cas normaux, limites et invalides
- utiliser des tests paramétrés avec @TestParametre
- regrouper des tests dans une suite de tests

Contexte

Vous travaillez sur une plateforme de boutique en ligne contenant **trois classes métier** :

- **Panier** — panier d'achat avec articles et codes de réduction
- **Article** — article du catalogue avec référence, nom et prix
- **ServiceCommande** — service de validation et création de commandes

Code fourni — Classes à ne pas modifier

Classe Article (classe métier)

Représente un article du catalogue de la boutique.

```
public class Article {
    private final String reference;
    private final String nom;
    private double prix;

    public Article(String reference, String nom, double prix) {
        if (reference == null || reference.isBlank())
            throw new IllegalArgumentException("La référence ne peut pas être vide");
        if (nom == null || nom.isBlank())
            throw new IllegalArgumentException("Le nom ne peut pas être vide");
        if (prix < 0)
            throw new IllegalArgumentException("Le prix ne peut pas être négatif");
        this.reference = reference;
        this.nom = nom;
        this.prix = prix;
    }

    public String getReference() { return reference; }
    public String getNom()       { return nom; }
    public double getPrix()      { return prix; }

    public void setPrix(double nouveauPrix) {
        if (nouveauPrix < 0)
            throw new IllegalArgumentException("Le prix ne peut pas être négatif");
        this.prix = nouveauPrix;
```

```
    }

    @Override
    public String toString() {
        return nom + " (" + reference + ") — " + prix + " €";
    }
}
```

Classe LigneCommande (classe métier)

Représente une ligne dans le panier : un article avec sa quantité.

```
public record LigneCommande(Article article, int quantite) {
    public LigneCommande {
        if (article == null)
            throw new IllegalArgumentException("L'article ne peut pas être null");
        if (quantite <= 0)
            throw new IllegalArgumentException("La quantité doit être positive");
    }

    public double sousTotal() {
        return article.getPrix() * quantite;
    }
}
```

Classe Panier

```
public class Panier {
    private List<LigneCommande> lignes = new ArrayList<>();
    private String codeReduction = null;

    public void ajouterArticle(Article article, int quantite) {
        if (article == null)
            throw new IllegalArgumentException("L'article ne peut pas être null");
        if (quantite <= 0)
            throw new IllegalArgumentException("La quantité doit être positive");
        lignes.add(new LigneCommande(article, quantite));
    }

    public void appliquerCodeReduction(String code) {
        if (code == null || code.isBlank())
            throw new IllegalArgumentException("Le code de réduction est invalide");
        this.codeReduction = code;
    }

    public double calculerTotal() {
        double sousTotal = lignes.stream()
            .mapToDouble(LigneCommande::sousTotal)
            .sum();
        if ("REDUC10".equals(codeReduction)) return sousTotal * 0.90;
        if ("REDUC20".equals(codeReduction)) return sousTotal * 0.80;
        return sousTotal;
    }
```

```
    public int nombreArticles()  { return lignes.size(); }
    public boolean estVide()     { return lignes.isEmpty(); }
    public List<LigneCommande> getLignes() { return lignes; }
}
```

Classe ServiceCommande

```
public class ServiceCommande {
    private final DepotStock depotStock;

    public ServiceCommande(DepotStock depotStock) {
        this.depotStock = depotStock;
    }

    public Commande passerCommande(Panier panier, String identifiantClient) {
        if (panier.estVide())
            throw new IllegalStateException("Impossible de commander : le panier est vide");
        if (identifiantClient == null || identifiantClient.isBlank())
            throw new IllegalArgumentException("L'identifiant client est invalide");
        for (LigneCommande ligne : panier.getLignes()) {
            int stockDisponible = depotStock.getStock(ligne.article().getReference());
            if (stockDisponible < ligne.quantite())
                throw new StockInsuffisantException(
                    "Stock insuffisant pour : " + ligne.article().getNom());
        }
        return new Commande(identifiantClient, panier.calculerTotal(), LocalDateTime.now());
    }
}
```

Interfaces et exceptions

```
// Interface du dépôt de stock (à implémenter en test)
public interface DepotStock {
    int getStock(String referenceArticle);
}

// Exception métier
public class StockInsuffisantException extends RuntimeException {
    public StockInsuffisantException(String message) { super(message); }
}

// Classe de commande créée par le service
public record Commande(String identifiantClient, double total, LocalDateTime dateCreation) {}
```

01 Premiers tests unitaires — Panier

Créer la classe de test **PanierTest** et écrire les premiers tests sur des cas valides.
Rappel de la structure **AAA** (Arranger · Agir · Affirmer) :

```java
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class PanierTest {

    @Test
    void ajouterArticleDeitAugmenterLeNombreDeArticles() {
        // Arranger
        Panier panier = new Panier();
        Article article = new Article("REF-001", "Stylo bleu", 1.50);

        // Agir
        panier.ajouterArticle(article, 2);

        // Affirmer
        assertEquals(1, panier.nombreArticles());
    }
}
```

✓	À réaliser
☐	**PanierTest** — créer la classe de test
☐	`ajouterArticleDeitAugmenterLeNombreDeArticles()` — assertEquals(1, panier.nombreArticles())
☐	`calculerTotalDoitRetournerLaSommeDessousTotaux()` — 2 articles à 1.50 € × 3 = 9.00 €
☐	`panierVideDoitRetournerEstVideEgalTrue()` — assertTrue(panier.estVide())
☐	`panierAvecArticlesNeDoitPasEtreVide()` — assertFalse(panier.estVide()) après ajout

02 Tests sur la classe Article

Avant de tester le **Panier**, écrivons des tests sur la classe métier **Article** elle-même.
C'est une bonne pratique : tester les briques de base avant de tester les composants qui les utilisent.

```java
class ArticleTest {

    @Test
    void creerArticleValideDoitFonctionner() {
        // Arranger & Agir
        Article article = new Article("REF-001", "Cahier", 3.50);

        // Affirmer
        assertEquals("REF-001", article.getReference());
        assertEquals("Cahier",  article.getNom());
        assertEquals(3.50,       article.getPrix(), 0.001);
    }
```

```
    @Test
    void modifierPrixDoitMettreAJourLePrix() {
        Article article = new Article("REF-002", "Règle", 1.00);
        article.setPrix(1.50);
        assertEquals(1.50, article.getPrix(), 0.001);
    }
}
```

✓	À réaliser
☐	Créer la classe **ArticleTest**
☐	`creerArticleValideDoitFonctionner()` — vérifier référence, nom et prix
☐	`modifierPrixDoitMettreAJourLePrix()` — setPrix(1.50) → getPrix() == 1.50
☐	`referenceNulleDoitLeverException()` — new Article(null, "Règle", 1.0) → IllegalArgumentException
☐	`nomVideDoitLeverException()` — new Article("REF-001", "", 1.0) → IllegalArgumentException
☐	`prixNegatifAlaCreationDoitLeverException()` — new Article("REF-001", "X", -1.0) → IllegalArgumentException
☐	`setPrixNegatifDoitLeverException()` — article.setPrix(-5.0) → IllegalArgumentException

03 Cas invalides — Panier

Écrire des tests qui vérifient que les **exceptions sont bien levées** pour les entrées incorrectes.

```
@Test
void quantiteNulleDoitLeverUneException() {
    Panier panier = new Panier();
    Article article = new Article("REF-001", "Stylo", 1.50);
    assertThrows(IllegalArgumentException.class,
            () -> panier.ajouterArticle(article, 0));
}
```

✓	À réaliser
☐	`articleNulDoitLeverException()` — ajouterArticle(null, 1)
☐	`quantiteNulleDoitLeverException()` — ajouterArticle(article, 0)
☐	`quantiteNegativeDoitLeverException()` — ajouterArticle(article, -3)
☐	`codeReductionVideDoitLeverException()` — appliquerCodeReduction("")
☐	`codeReductionNulDoitLeverException()` — appliquerCodeReduction(null)

04 Cas limites — Panier

Écrire des tests sur des **valeurs aux frontières** des règles métier.
Ce sont souvent les cas où les bugs se cachent : valeurs minimales, maximales, nulles.

```
@Test
void articleOfferDoitEtreAccepteEtNePasCompterDansLeTotal() {
    Panier panier = new Panier();
    Article articleGratuit = new Article("OFFERT-01", "Stylo offert", 0.0);
    panier.ajouterArticle(articleGratuit, 1);
    assertEquals(0.0, panier.calculerTotal(), 0.001);
}
```

✓	À réaliser
☐	`quantiteUneDoitEtreAcceptee()` — quantite = 1, prix = 9.99 → total = 9.99
☐	`articleGratuitDoitEtreAccepte()` — prix = 0.0, total = 0.0
☐	`prixEleveDoitFonctionner()` — prix = 999.99, quantite = 1 → total = 999.99
☐	`panierAvecUnSeulArticleDoitFonctionner()` — nombreArticles() == 1
☐	`plusieursArticlesDifferentsDansPanier()` — 3 articles différents → total = somme des sous-totaux

05 Tests sur ServiceCommande

Créer la classe de test **ServiceCommandeTest** avec un **double de test** pour DepotStock.

Rappel — Double de test (Stub)

Pour tester **ServiceCommande** sans base de données réelle, on crée une implémentation de test de **DepotStock** :

```
// Version classe nommée – plus lisible pour les débutants
class StockToujoursDisponible implements DepotStock {
    public int getStock(String reference) { return 100; }
}

// Version lambda – équivalente, plus concise
DepotStock stockSuffisant = reference -> 100;
```

```
class ServiceCommandeTest {
    private DepotStock stockDisponible = reference -> 100;
    private ServiceCommande service;
```

```
    private Panier panier;
    private Article articleTest;

    @BeforeEach
    void initialiser() {
        service     = new ServiceCommande(stockDisponible);
        panier      = new Panier();
        articleTest = new Article("REF-001", "Cahier", 3.50);
    }

    @Test
    void commandeValideDoitRetournerUneCommande() {
        panier.ajouterArticle(articleTest, 2);
        Commande commande = service.passerCommande(panier, "CLIENT-42");
        assertNotNull(commande);
        assertEquals(7.0, commande.total(), 0.001);
    }
}
```

✓	À réaliser
☐	`commandeValideDoitRetournerUneCommande()` — panier non vide, stock suffisant → Commande non nulle
☐	`panierVideDoitLeverIllegalStateException()` — passerCommande(panierVide, "C1") → IllegalStateException
☐	`identifiantClientNulDoitLeverException()` — passerCommande(panier, null) → IllegalArgumentException
☐	`identifiantClientVideDoitLeverException()` — passerCommande(panier, "") → IllegalArgumentException
☐	`stockInsuffisantDoitLeverStockInsuffisantException()` — quantite demandée > stock disponible → StockInsuffisantException
☐	`totalCommandeDoitCorrespondreAuTotalDuPanier()` — commande.total() == panier.calculerTotal()

06 Tests paramétrés — codes de réduction

Utiliser **@TestParametre** pour tester tous les scénarios de réduction en un seul test.

Les codes `REDUC10` (-10%) et `REDUC20` (-20%) sont des branches distinctes du code. Plutôt qu'écrire 3 tests séparés, on les paramètre.

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

class PanierReductionTest {

    @ParameterizedTest
    @CsvSource({
```

```
        "       ,  100.0",   // pas de code de réduction
        "REDUC10,   90.0",   // -10%
        "REDUC20,   80.0"    // -20%
    })
    void calculerTotalDoitAppliquerLaBonneReduction(
            String code, double totalAttendu) {
        // Arranger
        Panier panier = new Panier();
        Article article = new Article("REF-001", "Classeur", 10.0);
        panier.ajouterArticle(article, 10); // sous-total = 100.0

        // Agir
        if (code != null && !code.isBlank()) {
            panier.appliquerCodeReduction(code.trim());
        }

        // Affirmer
        assertEquals(totalAttendu, panier.calculerTotal(), 0.001);
    }
}
```

✓	À réaliser
☐	Créer **PanierReductionTest** avec le test paramétré ci-dessus
☐	Ajouter un jeu de données avec plusieurs articles (quantite > 1) pour chaque code
☐	Bonus — paramétrer les cas invalides de `ajouterArticle()` avec @CsvSource :
☐	paramétrer sur (référence, quantite, prix) → IllegalArgumentException attendue

07 Suite de tests

Objectif

Regrouper les classes de tests pour constituer une **campagne de tests automatisable**.
C'est utile pour :

- valider l'ensemble du module Panier & Commandes d'un seul clic
- distinguer les tests unitaires (rapides) des tests d'intégration
- organiser les campagnes CI (commit, nuit, mise en production)

7.1 — Projet minimum attendu

- `ArticleTest.java`
- `PanierTest.java`
- `PanierReductionTest.java`
- `ServiceCommandeTest.java`

7.2 — Créer la suite de tests

Créer la classe **SuiteBoutique** (sans méthode `@Test`) :

```
import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;

@Suite
@SelectClasses({
        ArticleTest.class,
        PanierTest.class,
        PanierReductionTest.class,
        ServiceCommandeTest.class
})
class SuiteBoutique {
    // Pas de méthode — JUnit exécute les classes déclarées
}
```

> Cette classe ne contient **aucune méthode de test** : elle sert uniquement à lancer plusieurs classes ensemble.

7.3 — Variante : suite par package

```
@Suite
@SelectPackages("fr.boutique.tests")
class TousBoutiquTests {}
```

Note : pour ce TP, la version avec @SelectClasses est plus simple et plus explicite.

✓	À réaliser
☐	Créer la classe **SuiteBoutique** avec `@SelectClasses`
☐	Inclure les 4 classes de tests
☐	Lancer la suite depuis l'IDE et vérifier que tous les tests passent au vert

Organisation attendue du projet

```
src
 ├── main/java/fr/boutique/
 │    ├── Article.java
 │    ├── LigneCommande.java
 │    ├── Panier.java
 │    ├── ServiceCommande.java
 │    ├── Commande.java
 │    ├── DepotStock.java
 │    └── StockInsuffisantException.java
 └── test/java/fr/boutique/
      ├── ArticleTest.java
      ├── PanierTest.java
      ├── PanierReductionTest.java
      ├── ServiceCommandeTest.java
```

```
    └── SuiteBoutique.java
```

Ce qu'ils doivent retenir

À la fin du TP, les étudiants doivent avoir compris que :

- **une classe de test** vérifie un composant (Article, Panier ou ServiceCommande)
- **une suite de tests** regroupe plusieurs classes pour constituer une campagne
- **les tests paramétrés** évitent la duplication (ex. : codes REDUC10, REDUC20…)
- **tester les classes métier** (Article, LigneCommande) avant les services est une bonne pratique
- **on ne cherche pas l'exhaustivité** mais la pertinence — 5 cas bien choisis valent mieux que 500